

JENA

Ulises Cortés, Luis Oliva Felipe

{ia,loliva}@cs.upc.edu

<http://www.cs.upc.edu/~{ia,loliva}>

SID2018

<https://goo.gl/VDV1dV>

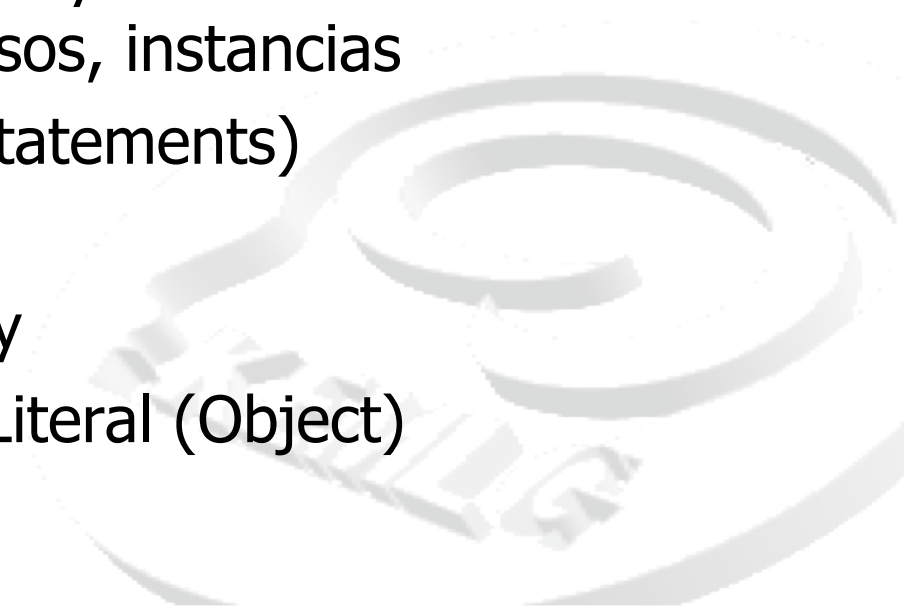
<https://kemlg.upc.edu>



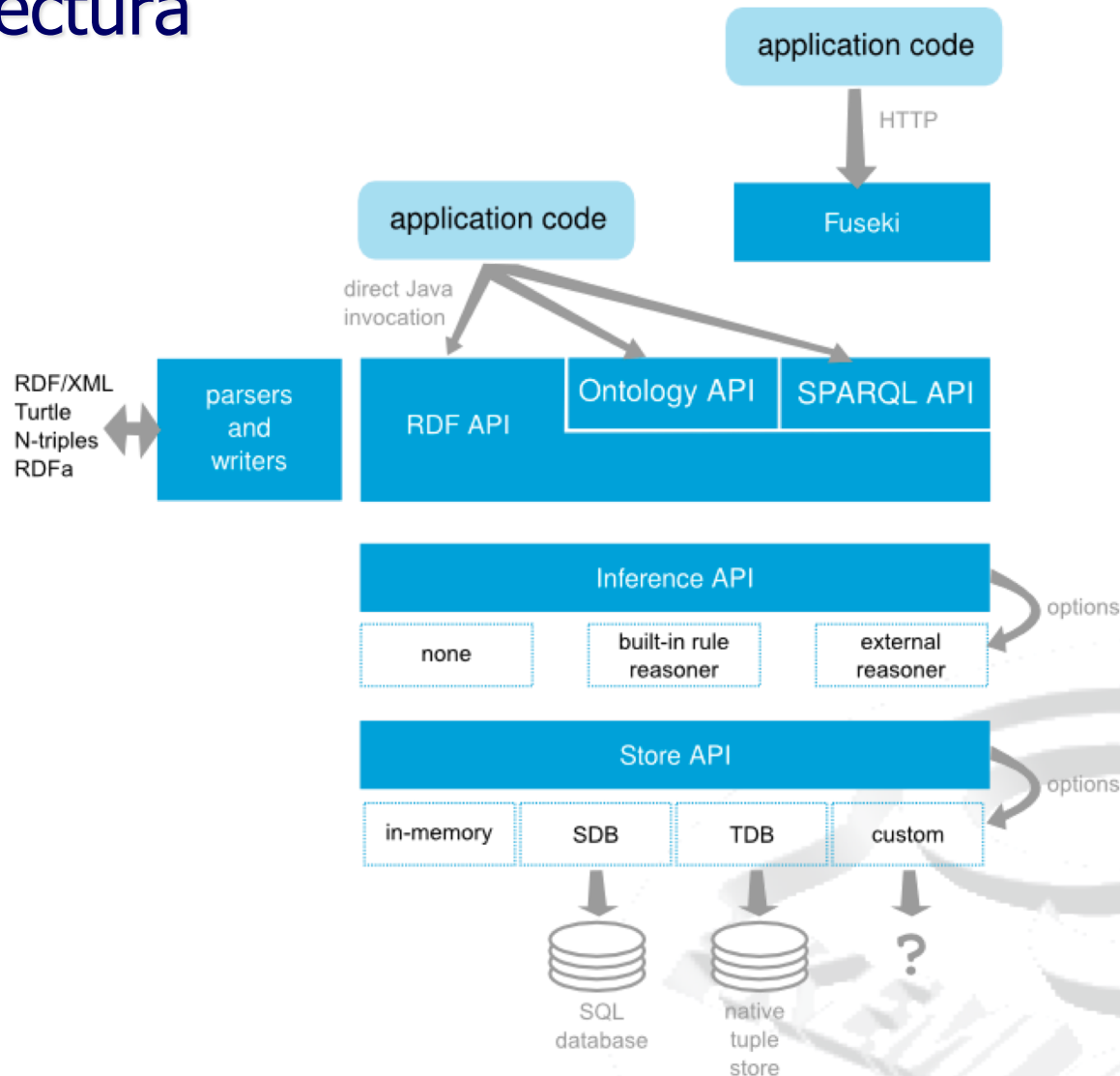


¿Qué és?

- API para gestionar (CRUD) grafos RDF
- Permite, programáticamente:
 - Crear/cargar ontologías (models)
 - Lanzar SPARQL
 - Añadir, modificar y eliminar elementos: propiedades, recursos, instancias
- Modelo de tripletas (statements)
 - Sujeto: Resource
 - Predicado: Property
 - Objeto: Resource/Literal (Object)



Arquitectura



Instalación

- Descargar apache_jena_3.6.0.zip (binary)
 - <https://jena.apache.org/download/index.cgi>
 - Incorporar a vuestro IDE (e.g., Eclipse)
 - Añadidlos a una librería propia (user library)
 - Adjuntad sources y javadoc para ayudaros
 - <https://goo.gl/YxyFRd>

- Para probar:

```
import org.apache.jena.rdf.model.*;
import org.apache.jena.datatypes.xsd.XSDDatatype;

public class HelloRDFWorld {
    public static void main(String[] args) {
        String NS = "http://example.com/";
        Model m = ModelFactory.createDefaultModel();
        Resource r = m.createResource (NS + "r");
        Property p = m.createProperty (NS + "p");
        r.addProperty(p, "Hello world", XSDDatatype.XSDstring);
        m.write(System.out, "Turtle");
    }
}
```



Carga de una ontología

- Creación de cero:

```
OntModel base = ModelFactory.createOntologyModel(OWL_DL_MEM_TRANS_INF);
```

```
// La entrada es un valor de OntModelSpec
```

```
Model base = ModelFactory.createDefaultModel();
```

- Carga desde un fichero

```
read( String url )  
read( Reader reader, String base )  
read( InputStream reader, String base )  
read( String url, String lang )  
read( Reader reader, String base, String Lang )  
read( InputStream reader, String base, String Lang )
```

- url: Para cargar de forma remota
- InputStream y Reader para cargarlo desde un fichero
- Base: para resolver URIs relativas (null)
- Lang: "Turtle", "RDF/XML", "N3"

Modos de carga

OntModelSpec	Language profile	Storage model	Reasoner
OWL_MEM	OWL full	in-memory	none
OWL_MEM_TRANS_INF	OWL full	in-memory	transitive class-hierarchy inference
OWL_MEM_RULE_INF	OWL full	in-memory	rule-based reasoner with OWL rules
OWL_MEM_MICRO_RULE_INF	OWL full	in-memory	optimised rule-based reasoner with OWL rules
OWL_MEM_MINI_RULE_INF	OWL full	in-memory	rule-based reasoner with subset of OWL rules
OWL_DL_MEM	OWL DL	in-memory	none
OWL_DL_MEM_RDFS_INF	OWL DL	in-memory	rule reasoner with RDFS-level entailment-rules
OWL_DL_MEM_TRANS_INF	OWL DL	in-memory	transitive class-hierarchy inference
OWL_DL_MEM_RULE_INF	OWL DL	in-memory	rule-based reasoner with OWL rules
OWL_LITE_MEM	OWL Lite	in-memory	none
OWL_LITE_MEM_TRANS_INF	OWL Lite	in-memory	transitive class-hierarchy inference
OWL_LITE_MEM_RDFS_INF	OWL Lite	in-memory	rule reasoner with RDFS-level entailment-rules
OWL_LITE_MEM_RULES_INF	OWL Lite	in-memory	rule-based reasoner with OWL rules
DAML_MEM	DAML+OIL	in-memory	none
DAML_MEM_TRANS_INF	DAML+OIL	in-memory	transitive class-hierarchy inference
DAML_MEM_RDFS_INF	DAML+OIL	in-memory	rule reasoner with RDFS-level entailment-rules
DAML_MEM_RULE_INF	DAML+OIL	in-memory	rule-based reasoner with DAML rules
RDFS_MEM	RDFS	in-memory	none
RDFS_MEM_TRANS_INF	RDFS	in-memory	transitive class-hierarchy inference
RDFS_MEM_RDFS_INF	RDFS	in-memory	rule reasoner with RDFS-level entailment-rules



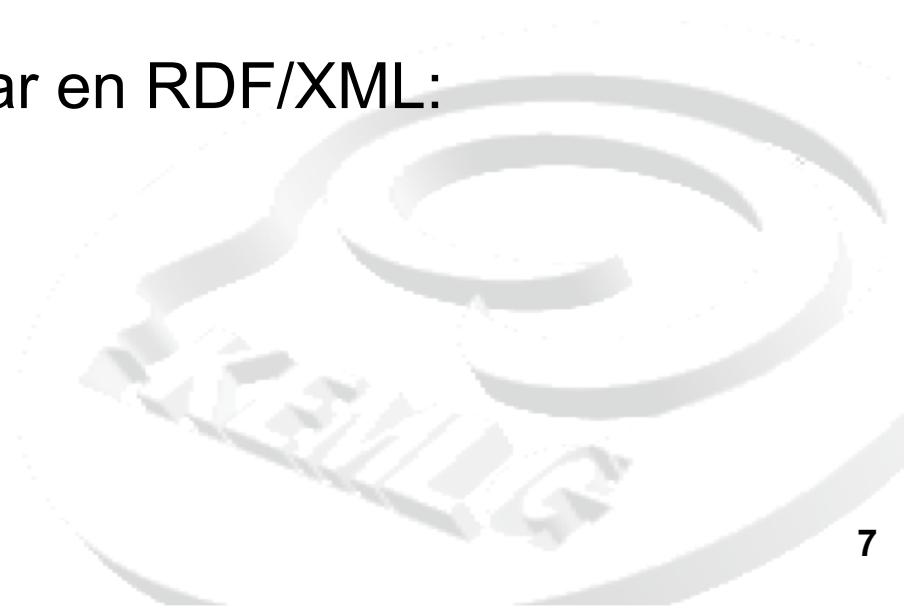
Guardando una ontología

```
write(Writer);
write(Writer, lang);
write(Writer, lang, base);
write(OutputStream);
write(OutputStream, lang);
write(OutputStream);
write(OutputStream, lang, base);
```

```
writeAll(Writer, lang);
writeAll(Writer, lang, base);
writeAll(OutputStream, lang);
writeAll(OutputStream, lang, base);
```

- Lo más básico, guardar en RDF/XML:

```
base.write(System.out);
```





Resources (OntResource y Resource)

- OntResource es una superclase de los elementos de la ontología
- Atributos comunes: versionInfo, comment, label, seeAlso , sameAs, differentFrom
- Para cada una hay los siguientes métodos:
 - add<property>
 - set<property>
 - list<property>
 - get<property>
 - has<property>
 - remove<property>
- También hay para rdf:type (cuidado)

Clases (OntClass)

- Creación:

```
OntClass person = m.createClass(NS+ "Person");
```

- Recuperación

```
Resource r = m.getResource(NS + "Person");  
OntClass person = r.as( OntClass.class);
```

```
OntClass c = m.getOntClass (NS + "Person");
```

- Atributos de OntClass:

- subClass
- superClass
- equivalentClass
- disjointWith

- Se puede hacer set, add, get, list, remove...

Properties (OntProperty)

- Tres tipos
 - ObjectProperty
 - DatatypeProperty
 - AnnotationProperty

```
OntClass person = m.createClass (NS + "Person");  
OntClass gender = m.createClass (NS + "Gender");
```

```
ObjectProperty hasGender = m.createObjectProperty(NS + "hasGender");
```

```
hasGender.addDomain(person);  
hasGender.addRange(gender);  
hasGender.addLabel( "has gender", "en");
```

```
DatatypeProperty birthDate = m.createDatatypeProperty(NS + "birthdate");  
birthDate.addDomain (m.getOntClass(NS + "Person"));  
birthDate.addRange(XSD.dateTime);
```



Resources (OntResource)

- Otros métodos
`getCardinality (Property p)`
- Borrar un recurso: `remove()`
- Dar valor a una propiedad dada:
`addPropertyValue(Property p, RDFNode value)`
- Obtener el valor de una propiedad dada:
`getPropertyValue(Property P)`
- Listar los tipos RDF de un recurso:
`listRDFTypes(boolean)`



Properties (OntProperty)

- Las properties tienen también como atributos:
 - subProperty
 - superProperty
 - Domain
 - Range
 - equivalentProperty
 - Inverse
- Como en los anteriores casos, se pueden gestionar con add, has, list, get
- Se puede convertir una property a un tipo más concreto mediante as():
 - asTransitiveProperty, asFunctionalProperty...

Instancias (individual)

- Creación

```
OntClass person = m.createClass (NS + "Person");
```

```
Individual i0 = m.createIndividual (NS + "JohnSmith", person);
```

```
Individual i1 = person.create(NS + "JohnSmith");
```

- Listado

```
person.listInstances();
```

```
person.listInstances(boolean direct);
```

- Eliminación

```
Person.dropIndividual(Resource ind); // removes rdf:type
```

```
ind.remove(); // To effectively remove the individual
```



Metadatos de la ontología

- Como en resources, una ontología tiene unos atributos:
 - `backwardCompatibleWith`
 - `incompatibleWith`
 - `priorVersion`
 - Imports
- Con métodos `add`, `set`, `get`, `list`, `test`, `delete`

```
Ontology ont = m.getOntology(baseURI);  
Ont.addProperty( DCTerms.creator, "John Smith");  
Ont.addImport(m.createResource(http://example.com/import1));
```



Razonador

- Ejemplo:

```
String NS = http://ejemplo.org/;  
Model m = ModelFactory.createDefaultModel();  
Property p = m.createProperty(NS, "p");  
Property q = m.createProperty(NS, "q");  
m.add(p, RDFS.subPropertyOf, q);  
m.createResource(NS+"a").addProperty(p, "foo");
```

```
InfModel inf = ModelFactory.createRDFSModel(m);
```

```
Resource a = inf.getResource(NS+"a");  
System.out.println(a.getProperty(q));
```

- Podemos usarlo para:

- Validar: `ValidityReport valid = infModel.validate();
validity.isValid(); validity.getReports();`

- Extender los statements



Razonador

- Podemos crear un modelo con base otro

```
OntModel base = ModelFactory.createOntologyModel(OWL_DL_MEM);
```

```
// La entrada es un valor de OntModelSpec
```

```
OntModel inf = ModelFactory.createOntologyModel(OWL_DL_MEM_TRANS_INF, base);
```

- De esta manera tenemos el modelo estático (base) y el modelo inferido (inf).



Lanzar queries (1/2)

- Con SPARQL

```

Import org.apache.jena.query;

String queryString = "PREFIX rdf... select ?x where {...}";
Query query = QueryFactory.create(queryString);

Try (QueryExecution qe = QueryExecutionFactory.create(query, model)){

    ResultSet results = qe.execSelect();
    // results = ResultSetFactory.copyResults(results);
    while(results.hasNext()){
        QuerySolution sol = results.next();
        RDFNode n = sol.get("varName");
        if (n.isResource()) { Resource r2 = (Resource) n;}
        if (n.isLiteral()) { Literal l2 = (Literal) n;} qe.close();
        Resource r = sol.getResource("VarR");
        Literal l = sol.getLiteral("VarL");
    }

    ResultSetFormatter.out(System.out, results, query)
    qe.close();
}
  
```

Lanzar queries (2/2)

- CONSTRUCT, DESCRIBE, ASK queries

```
Query query = QueryFactory(queryString);
```

```
QueryExecution qe = QueryExecutionFactory.create(query, model);
```

```
Model resultModel = qe.execConstruct();
```

```
Model resultModel = qe.execDescribe();
```

```
boolean result = qe.execAsk();
```

```
qe.close();
```

- Para añadir named graphs

```
String defaultModel = "file:default.ttl";
```

```
List named = new ArrayList();
```

```
named.add("file:named-1.ttl");
```

```
Dataset ds = DatasetFactory.create(defaultModel, named);
```

```
Dataset ds2 = DatasetFactory.create();
```

```
ds.setDefaultModel(model);
```

```
ds.addNamedModel("http://blah/named-1", model1);
```

```
try(QueryExecution qe = QueryExecutionFactory.create(query, model)){
```

```
    ...
```

```
}
```



ARQ

```
select * { ?person <http://xmlns.com/foaf/0.1/name> ?name }
```

```
QuerySolutionMap initialBinding = new QuerySolutionMap();
initialBinding.add("name", personResource); qe =
QueryExecutionFactory.create(query, dataset,
initialBinding);
```





Ejercicios

- Cargad pizza.owl
 - Escribid las super clases de una clase dada (subClassOf)
 - Determinad si una entidad dada es instancia de una clase dada (rdf type)
 - Determinad si dos clases dadas tienen el mismo padre
 - Arreglad el código de queries2.pizza
 - Añadid una propiedad hasPizzaName, Eaten
 - Comprobad que la función runSparqlQueryModify funciona bien haciendo que las pizzas se puedan comer
 - Comprobad que la función runSparqlQueryDataProperty permite recuperar pizzas con nombre
- Cread una ontología que modele masas de agua
 - Volumen, contaminantes
 - Diferentes tipos de masa de agua
 - Cread instancias de masa de agua y clasificadlas
 - Listad esas masas de agua junto con su clasificación



References

- <https://jena.apache.org/documentation/inference/index.html>
- <https://jena.apache.org/documentation/javadoc/>
- <https://jena.apache.org/documentation/ontology/>





Knowledge Engineering and Machine Learning Group UNIVERSITAT POLITÈCNICA DE CATALUNYA

Ulises Cortés, Luis Oliva Felipe

{ia,loliva}@lsi.upc.edu

<http://www.lsi.upc.edu/{ia,loliva}>

2018